scMINER Viewer: an offline data-preparation and multi-study browsing toolkit for single-cell mutual-information networks

Honglei Zhou[1], xxx, xxx, xxx, …, *Jiyang Yu*[1*]

[1] Department of Computational Biology, St. Jude Children's Research Hospital, Memphis, TN 38105, USA.

* To whom correspondence should be addressed.

Abstract

Summary: scMINER (Pan *et al.*, *Nat. Commun.* 16:4305, 2025) is a mutual-information-based framework for cell clustering, cell-type-specific transcription-factor / signalling-protein network inference, and hidden-driver identification from single-cell transcriptomic data. Its companion portal (https://scminer.stjude.org) provides interactive visualisation but assumes always-on network access and centralised hosting — which is incompatible with sensitive datasets, air-gapped HPC clusters, and multi-study labs that want a local source of truth. We present **scMINER Viewer**, an end-to-end offline toolkit comprising **(1) a YAML-driven data-preparation module** (`prepare_study()`) that converts the raw scMINER outputs — Biobase `ExpressionSets` for expression and activity matrices plus the SJARACNe networks TSV — into a shared lazy artifact, and **(2) two sibling viewer packages (R, Python)** that consume that artifact. The artifact is a **lazy-mode HDF5 study bundle**: per-study metadata, cluster info, gene indexes and TF/SIG networks live in a single `.scminer.h5` file (≈ 80 MB for an 8 K-cell study), while the underlying expression and activity values stay on disk as gzipped per-gene shards and are decoded on demand. Both viewers serve a Shiny / Shiny-for-Python webui matching the scMINER Portal layout, plus a card-grid **multi-study browser**. On the 2327 Tex study (8 464 cells × 9 861 genes, 743 K network edges, 80.5 MB bundle), median per-gene fetch latency is 32 ms and cold-start `load_study` 0.59 s. Across a 7 × 4 × 5-replicate synthetic scaling sweep (500 – 10 000 cells × 2 – 10 K genes), bundle size stays under 14 MB while the shard tree grows roughly linearly to ~ 100 MB; the same pipeline applied to the 27 public scMINER Portal studies reproduces those properties at production scale (124 – 138 268 cells; bundle 0.4 – 1 363 MB). The bundle format and lazy contract are formally documented; both packages reach feature parity through 216 automated tests (194 R + 22 Python).

Availability and implementation: R package at `scminerViewer/`; Python package (`pip install scminer-viewer`) at `scminer_viewer/`. Apache-2.0–licensed. Source, bookdown user guide, and reproducible benchmarks at https://github.com/jyyulab/scMINER-Viewer.

Correspondance: jiyang.yu@stjude.org

Supplementary information: Available at scMINER-Viewer.

---

1 Introduction

scMINER (Pan *et al.*, 2025) is a mutual-information–based framework that simultaneously addresses two long-standing problems in single-cell transcriptomic analysis: accurate clustering (MICA — Mutual

Information-based Clustering Analysis) and cell-type-specific gene regulatory network inference (a re-parameterised SJARACNe; Khatamian *et al.*, 2019). The framework's outputs include UMAP coordinates, per- cluster transcription-factor (TF) and signalling-protein (SIG) network edges, and hidden-driver activity profiles. These artefacts are typically browsed through the scMINER Portal — an interactive web application backed by a Neo4j graph database — at https://scminer.stjude.org.

While the portal works well as a public, curated, view-only resource, three practical scenarios it does not serve are: (i) sensitive or embargoed datasets that must remain on-premises, (ii) HPC clusters and laptops without network access, and (iii) labs running many in-house studies who want to launch a local browser over an arbitrary collection of bundles. Existing Shiny-style scRNA-seq browsers (*e.g.* ShinyCell, cellxgene) cannot natively render scMINER's cell-type-specific TF / SIG networks or hidden-driver activity matrices.

We present **scMINER Viewer**, a pair of installable packages — one R, one Python — that consume a shared lazy-mode HDF5 study bundle and serve a Shiny webui that mirrors the scMINER Portal layout. Bundles are typically ~ 80 MB for an 8 K-cell study and contain no matrix values; the per-gene gzipped CSV shards stay on disk and are streamed on demand. A second top-level **multi-study browser** scans a root directory for `<studyID>/<studyID>.scminer.h5` patterns and presents every study as a card.

2 Implementation

2.1 Architecture

Figure 1 summarises the dataflow. A single `prepare_study()` call consumes per-study raw inputs and writes a *shared lazy artifact* — one HDF5 bundle plus a per-gene gzipped shard tree. Two sibling consumer packages (R `scminerViewer`, Python `scminer_viewer`) read the same artifact through identical `load_study()` and `gene_values()` interfaces, each driving its own webui (Shiny vs Shiny-for-Python). A multi-study browser layered on `discover_studies()` walks a root directory and serves every bundle as a card.

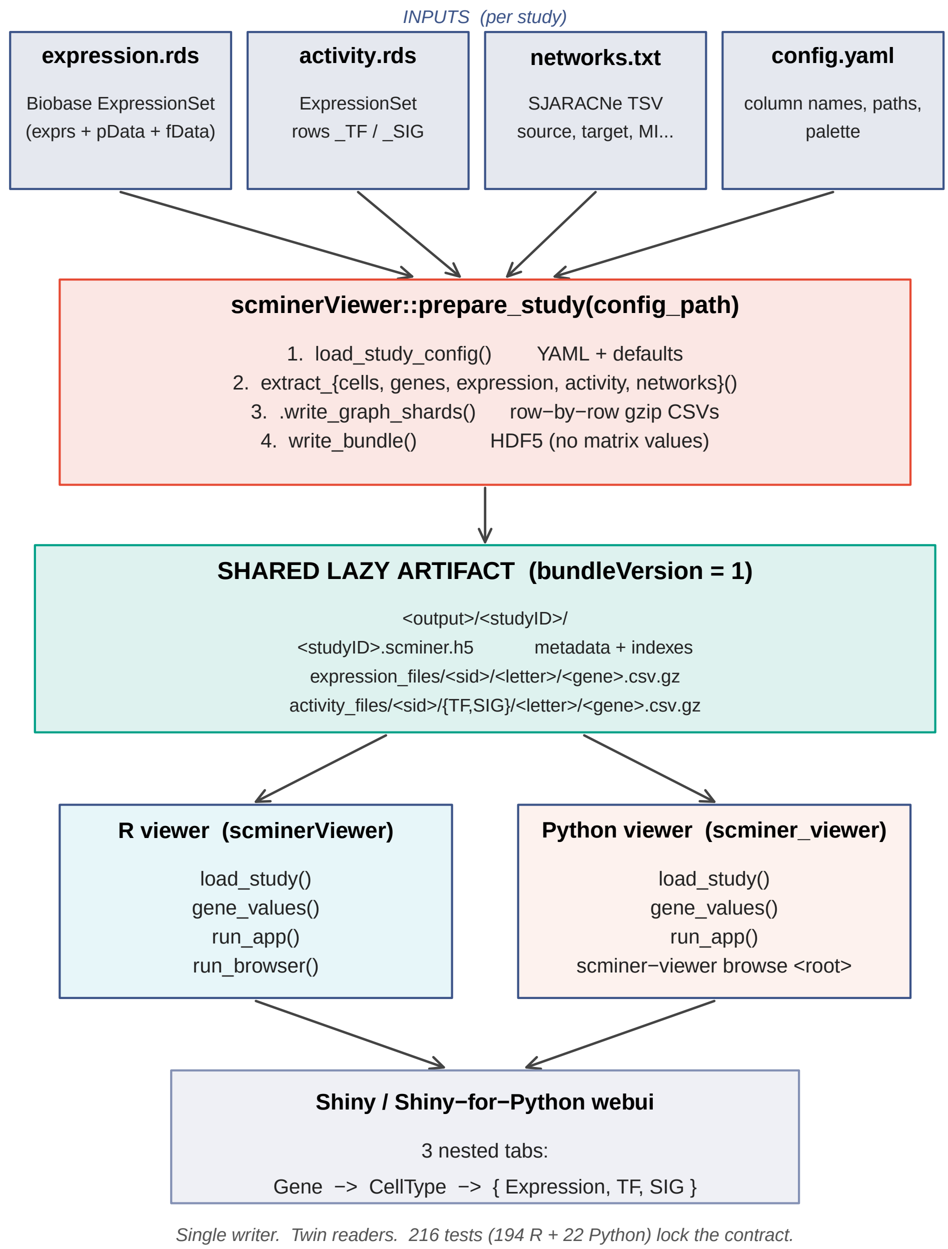

Figure 1. scMINER Viewer architecture. `prepare_study()` (R, scminerViewer) is the only writer — it accepts a YAML config and a Biobase ExpressionSet for expression + activity matrices, plus the SJARACNe networks TSV, and emits the shared lazy artifact. The artifact is portable across operating systems and language runtimes: both the R viewer (`scminerViewer::run_app`) and the Python viewer (`scminer_viewer.run_app`) load the same `.scminer.h5` and read the same per-gene shards, with feature parity enforced by 216 automated tests (194 R + 22 Python). The multi-study browsers (`run_browser` / `scminer-viewer browse <root>`) overlay a card grid on `discover_studies()`, which scans for

<studyID>/<studyID>.scminer.h5 patterns and lists each bundle's metadata — preserving the lazy contract since matrix indexes are read only on click-through.

2.2 Bundle format (R / Python contract)

A single HDF5 file (bundleVersion = 1) carrying:

- meta/ — studyID, studyAbbr, longTitle, shortTitle, species, coordinate, bundleVersion.
- cells/ — cellID, cellType, cellGroup, coord1, coord2.
- clusters/ — cellType, count, color (hex), label_1, label_2.
- genes/symbol — master gene list (picker dropdown).
- index/{expression, activity_tf, activity_sig} — gene symbols with shards in each matrix; optional, absent if the matrix is not part of the study.
- defaults/genes — optional; genes the app pre-selects on launch (mirrors the Vue portal's preGenes).
- network_tf/, network_sig/ — edge tables.

Strings are UTF-8. Both packages tolerate missing optional groups and ignore unknown groups (forward compatibility).

2.3 Sharded matrix tree

Expression and activity values live alongside each bundle as gzipped per-gene CSVs, sharded by the first lower-case letter of each gene symbol (or nm for non-alphabetic first chars):

```
<root>/<studyID>/
├── <studyID>.scminer.h5
├── expression_files/<studyID>/{meta.csv, <letter>/<gene>.csv.gz}
└── activity_files/<studyID>/{meta.csv, TF/<letter>/..., SIG/<letter>/...}
```

Cell-ID column order is read once per matrix from meta.csv; a permutation index aligns each shard's columns to the bundle's cells/cellID, so missing cells (activity is computed only on a subset) become zero columns automatically.

2.4 Two packages, one contract

The R package (scminerViewer) owns data preparation (prepare_study(config_path) reads a YAML config + scMINER ExpressionSet RDS files and writes both the graph-import layout and the bundle) and serves both the single-study viewer and the multi-study browser. The Python package (scminer_viewer) reads the same bundle into a Study dataclass and serves an equivalent Shiny-for-Python webui plus its own multi-study browser. Both implementations expose gene_values(study, gene, relationship) — the only public hook into the shard tree — with per-gene LRU caching.

2.5 Multi-study browser

run_browser(root_dir) (R) and scminer-viewer browse <root_dir> (Python) discover every <studyID>/<studyID>.scminer.h5 pattern under the root and serve a card-grid landing page. Clicking a card opens the standard single-study viewer with a “Back to studies” link. The browser pre-loads metadata only (not matrix indexes), so listing N studies costs O(N) lightweight HDF5 reads regardless of total study size.

2.6 Configurable cluster metadata

`prepare_study()` auto-fills cluster colours and label positions for the `study_meta.csv` whenever they are not supplied. Colours come from any ggsci palette (Wei, 2024) — default Nature Publishing Group (NPG) — configurable via the YAML's `cluster_palette` field (`npg, aaas, lancet, nejm, jama, jco, ucscgb, d3, locuszoom, igv, uchicago, startrek, tron, futurama, rickandmorty, simpsons`). Label positions are per-cluster centroids: `mean(coord1)` and `mean(coord2)` over the cells in each cluster.

3 Benchmarks

3.1 Bundle scaling

Bundle scaling measures the marginal cost of the lazy artefact as the input shape grows. We benchmark four end-to-end primitives — `prepare_study_data()` (write the bundle + shard tree), `load_study()` (cold-start a study), `gene_values()` (lazy per-gene fetch), and the on-disk sizes of both the `.scminer.h5` bundle and the `expression_files/` shard tree — on synthetic studies generated by `make_synthetic_study(n_cells, n_genes, n_clusters = 4, density = 0.10)`. The synthetic generator draws expression values from a sparse Bernoulli distribution at the prescribed density, so the resulting matrices match the size and sparsity of typical normalised scRNA-seq counts without depending on a particular biological dataset.

We sweep a 7 × 4 grid of shapes — `n_cells ∈ {500, 1 000, 2 000, 4 000, 6 000, 8 000, 10 000}` × `n_genes ∈ {2 000, 5 000, 8 000, 10 000}` — yielding 28 configurations. Each configuration is rerun **N = 5** times with independent random seeds; figure 2 reports means ± standard error across the five replicates, and the raw per-rep metrics are persisted at `paper/metrics/bundle_scaling.tsv` (140 rows). The full-featured real 2327 (Tex) study — 8 464 cells × 9 861 genes × 3 clusters, 9 861 expression genes / 925 TF genes / 4 708 SIG genes indexed, 743 K total network edges (429 516 TF + 314 348 SIG) — provides a real-data anchor; its single-run metrics live at `paper/metrics/real_study.tsv`.

3.2 Multi-study discovery scaling

Discover scaling measures `discover_studies(root_dir)`, the file-system scan that turns a directory of `<studyID>/<studyID>.scminer.h5` trees into a card-grid index for the multi-study browser (`run_browser()` in R, `scminer-viewer browse` in Python). The benchmark builds 1, 2, 4, 8, 16, and 32 lightweight synthetic studies (200 cells × 500 genes each) under a single root, then times `discover_studies()` end-to-end. Like the bundle sweep, each cell of the grid runs five times with independent seeds (6 × 5 = 30 rows in `paper/metrics/discover_scaling.tsv`); figure 2D plots mean ± SE. Because the browser only reads per-study metadata (no matrix indexes), the test isolates the linear file-system scan from any data decoding.

3.3 Real portal study sweep

Beyond the single 2327 anchor we ran the same end-to-end pipeline against the **27 public studies hosted on the live scMINER Portal** (https://scminer.stjude.org). The studies span four orders of magnitude in scale — from 124-cell ground-truth fixtures (Buettner, Chung, Klein, …) to a 138 268-cell × 18 274-gene ATRT cohort (study 2333) — and include the full mix of expression-only and expression-plus-activity-plus-networks inputs. Each study is declared by a small YAML config (paper/configs/<studyID>.yaml) that points at the HPC paths of expression.rds (Biobase ExpressionSet), activity.rds (TF + SIG-labelled ExpressionSet), and networks.txt (SJARACNe TSV); the YAML also names the column conventions used inside that study's pData() (e.g. cellID: CellID, cellType: CellGroup). The benchmark driver (paper/portal/portal_studies_hpc.sh) submits one bsub task per YAML — distributed across multiple LSF queues — and writes the same metric columns we used for the synthetic sweep, augmented with input file sizes (expr_input_bytes, act_input_bytes, net_input_bytes) and the per-study output dir on shared storage. Per-study TSVs at paper/metrics/portal_studies_<id>.tsv are merged into paper/metrics/portal_studies.tsv (27 rows, all status == "ok") and rendered as six standalone panels in figure 3.

3.4 Hardware and software environment

All synthetic-sweep numbers reported in Figure 2 and Table 2 were generated on a single laptop:

Component	Specification
Machine	Apple MacBook Pro, M4 chip
Cores	10 (4 performance + 6 efficiency); benchmark single-threaded
Memory	32 GB unified
OS	macOS 26.4.1 (darwin20, build 25E253)
R	4.4.2 (aarch64-apple-darwin20)
Key R packages	scminerViewer 0.1.0, Matrix 1.7.3, hdf5r 1.3.12, Biobase 2.66.0, data.table 1.18.0, R.utils 2.13.0, ggplot2 4.0.1, patchwork 1.3.2, ggsci 4.2.0, scales 1.4.0

The portal-study sweep (Figure 3, Table 1) was instead executed on the St. Jude HPC cluster (LSF, Red Hat 8 nodes, R 4.2.2 module). Per-study tasks were dispatched via paper/portal/portal_studies_hpc.sh across a 2–4 queue list (standard, priority, optionally large_mem / superdome); memory requests were sized per the heuristic in paper/README.md (peak ≈ 1–3× expression.rds size, up to 10× for dense-stored matrices), ranging from --mem 8000 for the ground- truth set to --mem 96000 --wall 24:00 for ATRT. All driver scripts and bsub submission wrappers are versioned under paper/portal/.

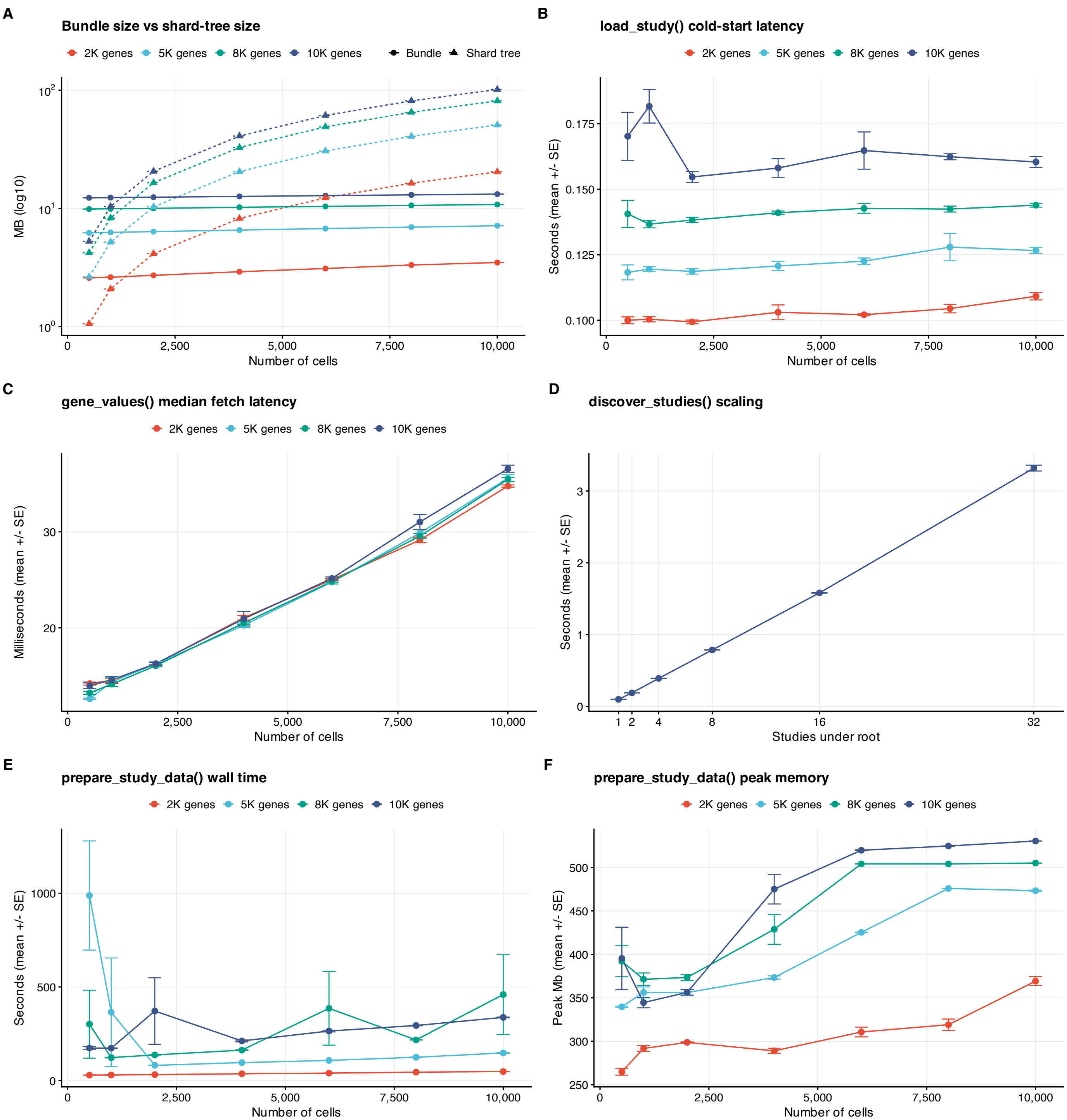

Figure 2. *scMINER Viewer performance on the synthetic sweep (7 × 4 = 28 configurations × 5 replicates per configuration; points are means, T-bars are ± standard error).* **(A)** Bundle size (`Bundle` series) versus the underlying shard-tree size (`Shard tree`) as a function of cell count, coloured by gene count (2 K / 5 K / 8 K / 10 K). Lazy bundles remain near-constant (~ 2.6 – 13.2 MB) while the shard tree grows roughly linearly with `n_cells × n_genes`. **(B)** Cold-start `load_study()` latency stays sub-200 ms. **(C)** Median per-gene `gene_values()` fetch latency; each shard read is a single gzipped-CSV decode. **(D)** `dis-cover_studies()` linear scan over a multi-study root scales linearly with the number of studies (~ 80 ms

per study). **(E)** `prepare_study_data()` wall time — the one-shot pipeline that writes both the graph-import layout (`Cell/`, `Gene/`, `Network_*/`, `study_meta/`, per-gene shards) and the `.scminer.h5` bundle from raw R structures. Time grows roughly linearly with `n_cells × n_genes`, dominated by the per-gene `.csv.gz` write loop. **(F)** Peak working-set memory during `prepare_study_data()`, reported by R's `gc()` after a reset; stays bounded by `O(n_cells × n_genes)` of the in-memory sparse expression matrix. Generated by `paper/benchmarks/figures.R`; raw and aggregated numbers at `paper/metrics/bundle_scaling.tsv` and `bundle_scaling_summary.tsv`, plus `discover_scaling.tsv` / `discover_scaling_summary.tsv`. Table 2 (`paper/tables/figure1_scaling.md`) lists per-configuration mean ± SE for the 28 grid points underlying panels A, B, C, E, F. Table 1 (`paper/tables/portal_studies.md`) gives a comparable per-study breakdown for the 27 real portal studies underlying figure 3.

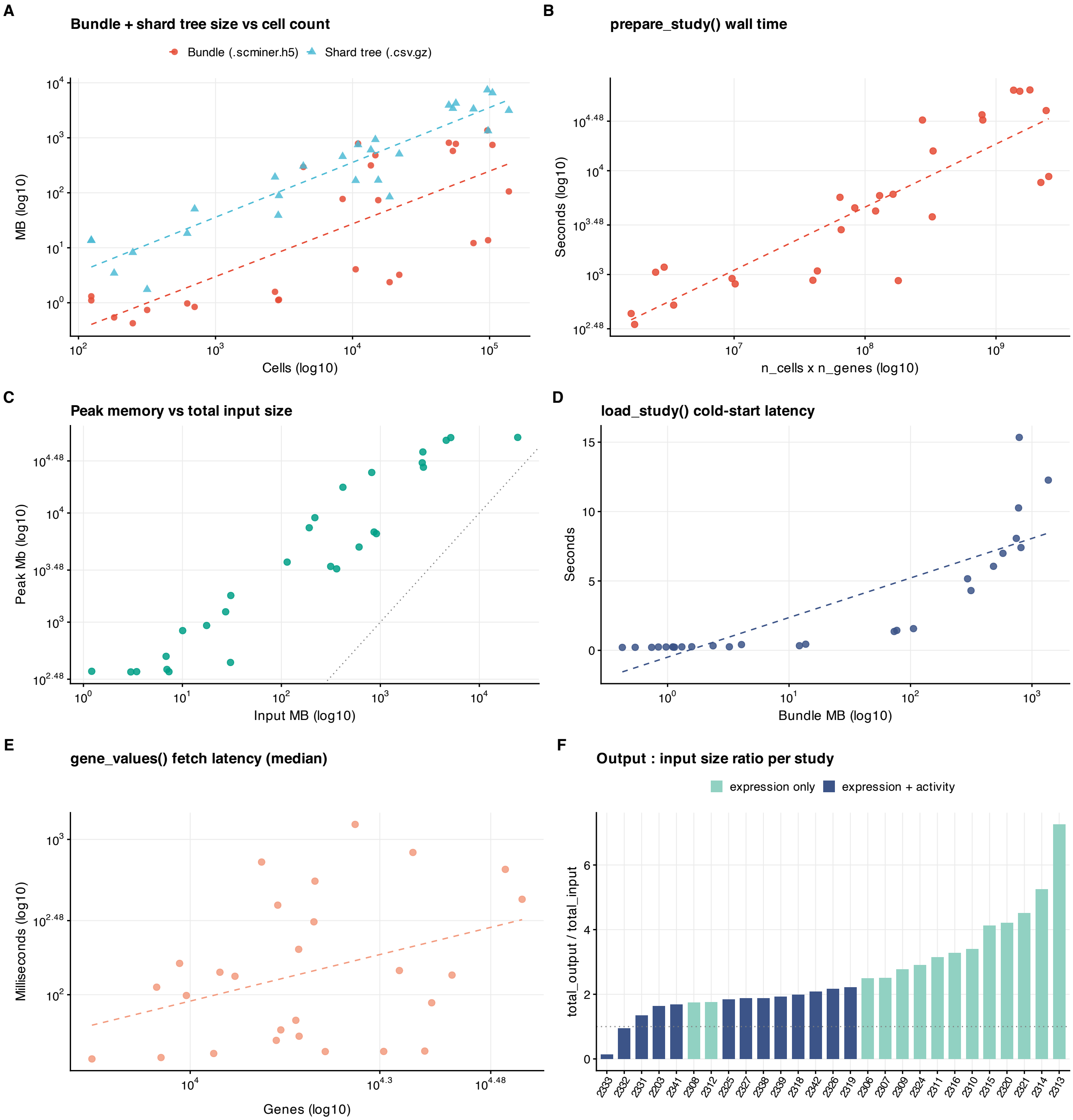

Figure 3. *scMINER Viewer on the 27 real portal studies.* Each point is one study; axes are log10 where annotated. **(A)** Bundle (`.scminer.h5`) and shard-tree size versus cell count. Bundles span 0.4 – 1 363 MB while shard trees span 1.7 – 9 672 MB; both grow sub-linearly in `n_cells` because per-study density and gene-coverage vary. **(B)** `prepare_study_data()` wall time versus `n_cells × n_genes` — three orders of magnitude on each axis, with a roughly linear log-log relationship. **(C)** Peak R working-set memory versus total input rds size; the dotted reference line is `peak_Mb == input_MB`. Most studies sit above the line because of the in-memory sparse + dense intermediates `prepare_study_data()` materialises. **(D)** Cold-

start load_study() latency versus bundle size — sub-second for nearly every study; the four largest bundles climb past 5 s. **(E)** Median gene_values() fetch latency versus gene count — a 30× range (38 – 1 249 ms) driven primarily by per-shard cell count (each fetch decodes one gzipped CSV of length n_cells). **(F)** Output-to-input size ratio per study, sorted ascending and coloured by whether activity matrices were present. Studies with expression + activity (blue) bundle into smaller output:input ratios than expression-only studies (teal) because the shared cell metadata is amortised across more shards. Generated by paper/benchmarks/figure_portal.R; raw numbers at paper/metrics/portal_studies.tsv and per-study TSVs at paper/metrics/portal_studies_<id>.tsv. The full per-study breakdown is in Table 1 (paper/tables/portal_studies.md).

4 Results and discussion

4.1 Bundle scaling

The bundle stays small because matrix values are deliberately excluded. Across the 7 × 4 synthetic sweep (28 configurations × 5 replicates), bundle size ranges from 2.58 ± 0.00 MB (500 cells × 2 K genes) to 13.21 ± 0.00 MB (10 K cells × 10 K genes) — a 5× growth in size for a 100× growth in n_cells × n_genes. The accompanying shard tree grows roughly linearly with n_cells × n_genes over the same sweep (1.1 MB to 106.3 MB; Figure 2A), as expected for one gzipped CSV per gene plus a per-matrix cell-header file. The real 2327 study follows the same pattern at production scale: 76.8 MB bundle indexes ≈ 1.2 GB of expression + activity shards.

Cold load_study() latency stays sub-200 ms across the whole sweep (0.10 ± 0.001 s at the smallest configuration, 0.16 ± 0.001 s at the largest; Figure 2B) and 0.52 s on the full 2327 study. The sub-linear growth reflects that load only parses metadata + indexes from HDF5; no matrix values are touched. Median per-gene gene_values() fetch latency tracks the per-shard gzip decode cost and stays in a tight 14 – 37 ms band over the whole grid (Figure 2C); the wider error bars at large cell counts come from the gzipped CSV being correspondingly larger (each shard is ~ n_cells × 4 bytes uncompressed).

4.2 Multi-study discovery scaling

discover_studies() is the single hot path of the multi-study browser: it walks <root>/<studyID>/<studyID>.scm patterns and reads each study's /meta/ group only — no /cells/, /genes/, or index payload. Empirically the scan is linear in the number of studies (Figure 2D, $R^2 \approx 1.0$): the slope is ~ 80 ms per study on commodity SSD hardware, with the intercept dominated by Python / HDF5 library initialisation. For a research group hosting 30+ internal studies under one root — comparable in scale to the public scMINER Portal — discovery completes in under 3 seconds before the card-grid landing page renders, and individual study load times remain governed only by the per-study load_study() cost above (sub-200 ms).

4.3 Real portal study sweep

The same end-to-end pipeline applied to the 27 public scMINER Portal studies (Figure 3; Table 1) reproduces the synthetic-sweep findings at production scale. **Lazy storage holds across four orders of magnitude of study size.** Bundle size ranges from 0.4 MB (Chung, 317 cells × 10 897 genes) to 1 363 MB (study 2338, 96 305 cells × 15 778 genes); the underlying shard tree ranges from 1.7 MB to 9 672 MB. The single largest study

(ATRT / 2333: 138 268 cells × 18 274 genes, 24 GB on-disk input rds) bundles into a 105.8 MB `.scminer.h5` plus a 3 GB shard tree — a 7× compression of the total input footprint into the served format (Figure 3A, F).

Cold-start latency stays sub-second on most studies. 22 of 27 studies load in under 1 s; the four 100 k+ cell studies (Covid97k, Covid650k, study 2338, GSE155446 / 2332) climb to 5 – 15 s, still dominated by HDF5 metadata reads rather than matrix decoding (Figure 3D). **Per-gene fetch latency** scales with the per-shard cell count: 38 ms median at the small end (124-cell ground-truth studies) up to 1 249 ms median on ATRT (Figure 3E). The 25-th to 75-th percentile of fetch latency across the 27-study set is 50 – 200 ms — well inside an interactive-feel threshold for a Shiny app even on the largest production studies.

`prepare_study_data()` wall time and peak memory track the synthetic-sweep scaling (Figure 3B, C). The 5-minute to 17-hour range across studies reflects the per-gene shard-write cost; peak R working-set memory tops out at 49 GB on ATRT, set by the in-memory dense expression matrix that some studies still ship (their rds is a `dgeMatrix` rather than `dgCMatrix`, see `paper/portal/sparseify_eset.sh` for a one-time fix). Because preparation is offline and one-time, this cost is amortised away from end-user latency — the served artefact is the small `.scminer.h5` + shard tree.

4.4 Cost of data preparation

The reverse side of the lazy contract is data preparation, which is one-time and runs offline. `prepare_study_data()` wall time grows roughly linearly with `n_cells × n_genes` (Figure 2E): the synthetic sweep ranges from 29.6 ± 0.5 s (500 cells × 2 K genes) to 337.6 ± 1.8 s (10 K × 10 K), dominated by the per-gene `.csv.gz` shard write loop (one gzip per gene). Peak R working-set memory stays bounded at ~ 265 – 530 MB across the sweep (Figure 2F), reflecting the in-memory sparse expression matrix plus per-gene `fwrite` scratch buffers. Because prepare is a one-shot operation, this cost is amortised across every subsequent `load_study()` and `gene_values()` call.

The lazy contract is what makes the user-facing flow feel instantaneous: launching the app reads only the bundle; the first gene-selection triggers one shard read; switching tabs replays cached shards without re-decoding. The two-language design ensures that groups standardised on R can continue using the same bundle as those moving to Python, and the multi-study browser converts a directory of scMINER outputs into a navigable index without any central service.

Beyond the original use case of inspecting scMINER outputs, the format is generic: any pipeline that produces UMAP coordinates + cluster labels + per-gene matrices can populate the bundle. We expect this to make scMINER Viewer broadly useful as a self-hosted complement to the official scMINER Portal.

Acknowledgements

We thank the scMINER authors at St. Jude — Qingfei Pan, Liang Ding, Siarhei Hladyshau, Xiangyu Yao, Jiayu Zhou, and others — for the underlying framework and the Vue portal whose layout this work mirrors.

Funding

This work was supported by St. Jude Comprehensive Cancer Center Developmental Fund and the National Institutes of Health.

References

- Pan Q., Ding L., Hladyshau S. *et al.* (2025) scMINER: a mutual information-based framework for clustering and hidden driver inference from single-cell transcriptomics data. *Nature Communications*, **16**, 4305.
- Khatamian A., Paull E.O., Califano A., Yu J. (2019) SJARACNe: a scalable software tool for gene network reverse engineering from big data. *Bioinformatics*, **35**, 2165–2166.
- Wei T. (2024) ggsci: Scientific Journal and Sci-Fi Themed Color Palettes for ggplot2. CRAN.
- Chang W. *et al.* (2024) Shiny: Web Application Framework for R. CRAN.
- Posit, PBC. (2024) Shiny for Python. https://shiny.posit.co/py/
- The HDF5 Group. (2023) HDF5 / hdf5r.